

Projet Mini-Blog :

Conception et réalisation d'une application web complète

Dossier d'architecture logicielle, choix de conception et implémentation technique

Fiche Documentaire

Projet : Mini-Blog

Auteur : Florian Pépin

Version : 1.0

Date : 25 septembre 2026

Table des matières

1	Introduction Générale et Périmètre du Projet	2
2	Partie I : Architecture et Réalisation du Backend	3
2.1	Choix de Conception et Justifications Techniques	3
2.1.1	Axe 1 : Mécanisme d'Authentification et Gestion de Session	3
2.1.2	Axe 2 : Inscription et Création du Premier Administrateur	3
2.1.3	Axe 3 : Règles de Publication et Modération des Commentaires	3
2.1.4	Axe 4 : Cycle de Vie et Modification des Articles	4
2.1.5	Axe 5 : Déclenchement de la Publication	4
2.1.6	Axe 6 : Confidentialité et Visibilité des Brouillons	4
2.1.7	Axe 7 : Suppression des Données et Liens en Cascade	5
2.1.8	Axe 8 : Évolution et Migrations de la Base de Données	5
2.1.9	Axe 9 : Conteneurisation et Gestion des Secrets	5
2.2	Modèle de Données et Schéma Relationnel	6
2.3	Matrice de Sécurité et Droits d'Accès	7
2.4	Architecture Logicielle en Couches	7
2.5	Validation et Tests Automatisés du Backend	7
3	Partie II : Architecture et Réalisation du Frontend	7
3.1	Choix de Conception du Client Web	7
3.1.1	Axe 10 : Solution de Style et Design Visuel	8
3.1.2	Axe 11 : Gestion des Pannes Réseau et Arrêt du Serveur	8
3.1.3	Axe 12 : Routage et Communication avec l'API	9
3.1.4	Axe 13 : Rédaction et Mise en Page des Articles	9
3.2	Architecture Logicielle et Découpage Modulaire	9
3.3	Écrans et Parcours Utilisateur	10
4	Partie III : Déploiement, Gestion des Secrets et Conclusion	10
4.1	Orchestration Conteneurisée et Isolation des Secrets	10
4.2	Conclusion et Bilan de Réalisation	11

1 Introduction Générale et Périmètre du Projet

Le projet **Mini-Blog** a pour objectif d'offrir une plateforme moderne, performante et sécurisée de publication et d'interaction communautaire. L'application est architecturée de la façon suivante :

- **Backend RESTful** : Développé en Java 21 LTS avec **Spring Boot 3**, assurant la gestion des utilisateurs, des articles, des commentaires, du hachage BCrypt, des jetons JWT stateless et du contrôle d'accès au niveau ressource.
- **Base de Données Relationnelle** : Propulsée par **PostgreSQL 16**, avec contrôle strict de schéma et migrations SQL versionnées via **Flyway**.
- **Frontend Single Page Application (SPA)** : Développé avec **Angular 18** en architecture *Standalone* intégrale, exploitant les **Signals** réactifs et le design system utilitaire **Tailwind CSS**.
- **Infrastructure & Conteneurisation** : Déploiement automatisé multi-conteneurs via **Docker Compose** avec serveur web **Nginx** servant l'application sur le port conventionnel 4200 et assurant le reverse-proxying interne vers l'API sans friction CORS.

2 Partie I : Architecture et Réalisation du Backend

2.1 Choix de Conception et Justifications Techniques

Cette sous-section détaille les décisions structurantes du backend.

2.1.1 Axe 1 : Mécanisme d'Authentification et Gestion de Session

La question : Comment reconnaître un utilisateur connecté et sécuriser ses requêtes sans alourdir le serveur ?

Options étudiées :

1. **Jeton numérique autonome (JWT stateless dans l'en-tête Authorization)** (Retenu).
2. Jeton stocké dans un cookie de session de navigateur.
3. Session classique où le serveur garde en mémoire tous les utilisateurs connectés.

Notre choix et pourquoi : Avec une session classique, le serveur doit mémoriser chaque internaute connecté, ce qui consomme de la mémoire et complique les montées en charge. Avec un jeton (JWT), c'est l'inverse : dès que l'utilisateur s'identifie, le serveur lui remet un « badge numérique chiffré » contenant son identifiant et son rôle. À chaque clic ou action, le navigateur présente ce badge. Le serveur n'a qu'à vérifier sa signature sans rien stocker. C'est plus léger, plus rapide, et c'est la façon la plus naturelle de faire communiquer une application Angular avec une API.

2.1.2 Axe 2 : Inscription et Création du Premier Administrateur

La question : Comment permettre aux visiteurs de s'inscrire tout en garantissant qu'un administrateur existe dès le départ ?

Options étudiées :

1. **Inscription publique pour les utilisateurs réguliers + Création automatique du premier administrateur au démarrage** (Retenu).
2. Inscription publique pour tous, puis promotion manuelle d'un compte en modifiant la base de données.
3. Inscription fermée (seul un administrateur peut créer de nouveaux comptes).

Notre choix et pourquoi : N'importe quel visiteur peut s'inscrire librement pour commencer à rédiger des articles. En revanche, le rôle d'administrateur donne tous les pouvoirs sur le site et ne doit jamais être accessible lors d'une inscription publique. Pour que la plateforme soit utilisable immédiatement dès le premier lancement sans exiger de manipulation manuelle dans la base de données, le backend crée automatiquement un compte administrateur par défaut s'il n'existe pas encore.

2.1.3 Axe 3 : Règles de Publication et Modération des Commentaires

La question : Qui a le droit de commenter et comment assurer la modération des discussions ?

Options étudiées :

1. **Commentaires réservés aux utilisateurs connectés sur articles publiés ; modification et suppression par l'auteur ; modération totale par l'administrateur** (Retenu).
2. Commentaires ouverts aux visiteurs anonymes sans compte.
3. Commentaires définitifs (impossibles à modifier une fois postés).

Notre choix et pourquoi : Ouvrir les commentaires aux anonymes attirerait inévitablement du spam et des messages indésirables : imposer d'être connecté responsabilise les auteurs. De plus, il est illogique de pouvoir commenter un article encore en brouillon : les discussions ne s'ouvrent que sur les articles publics. Enfin, pour le confort d'usage, chaque utilisateur peut corriger ou retirer ses propres messages, tandis que l'administrateur conserve le droit de supprimer n'importe quel commentaire pour faire respecter la charte du blog.

2.1.4 Axe 4 : Cycle de Vie et Modification des Articles

La question : Un auteur peut-il encore modifier ou supprimer son article une fois que l'administrateur l'a validé et publié ?

Options étudiées :

1. **Politique stricte : l'auteur ne peut retoucher son article que tant qu'il est en brouillon. Une fois publié, seul l'administrateur peut le modifier, le dépúblier ou le supprimer** (Retenu).
2. L'auteur peut modifier son article en ligne à tout moment sans révision.
3. Toute modification par l'auteur repasse automatiquement l'article en brouillon masqué.

Notre choix et pourquoi : C'est une règle de bon sens éditorial. Si un auteur pouvait modifier son texte librement après validation, il pourrait faire valider un article irréprochable par l'administrateur, puis le remplacer une heure plus tard par du contenu offensant ou publicitaire. Dès lors qu'un article est public, c'est l'administrateur qui engage la responsabilité du site : l'article est donc verrouillé pour l'auteur. Si un changement est requis, l'administrateur peut s'en charger ou renvoyer l'article en brouillon pour réécriture.

2.1.5 Axe 5 : Déclenchement de la Publication

La question : Comment concevoir l'action de publication dans l'application pour éviter les erreurs et les fraudes ?

Options étudiées :

1. **Boutons et actions dédiés réservés aux administrateurs (publish et unpublish)** (Retenu).
2. Un simple champ de formulaire à cocher lors de la modification de l'article.

Notre choix et pourquoi : Séparer nettement la modification du texte de l'action de validation évite toute confusion. Cela empêche surtout un utilisateur malveillant de glisser en douce une instruction « publier » lors de l'enregistrement ordinaire de son brouillon. Seuls les boutons d'action réservés aux administrateurs peuvent faire basculer un article d'un état à l'autre.

2.1.6 Axe 6 : Confidentialité et Visibilité des Brouillons

La question : Qui doit pouvoir voir les articles en cours de rédaction ?

Options étudiées :

1. **Filtrage intelligent : les visiteurs ne voient que les articles publiés, l'auteur ne voit que ses propres brouillons, et l'administrateur voit tout** (Retenu).
2. Créer plusieurs adresses web séparées selon qui regarde.

Notre choix et pourquoi : Un brouillon est un travail inachevé, personnel et confidentiel. Il est donc logique qu'aucun autre utilisateur ne puisse l'apercevoir tant qu'il n'est pas prêt. L'administrateur, lui, doit pouvoir accéder à l'ensemble des brouillons pour relire les propositions et décider de leur mise en ligne. Le système applique cette règle de confidentialité automatiquement en fonction de la personne connectée.

2.1.7 Axe 7 : Suppression des Données et Liens en Cascade

La question : Que doit-il se passer pour les commentaires lorsqu'un article est supprimé ?

Options étudiées :

1. **Suppression totale et propre : supprimer l'article efface automatiquement tous ses commentaires** (Retenu).
2. Conserver les commentaires orphelins sans leur article.
3. Masquer l'article sans jamais réellement effacer les données de la base.

Notre choix et pourquoi : Conserver des commentaires attachés à un article qui n'existe plus encombrerait inutilement la base de données et créerait des incohérences. Supprimer l'article et ses commentaires en un seul geste garantit que la base de données reste parfaitement propre, tout en respectant le droit à l'effacement.

2.1.8 Axe 8 : Évolution et Migrations de la Base de Données

La question : Comment s'assurer que la base de données s'installe à l'identique sur tous les ordinateurs sans risque d'erreur humaine ?

Options étudiées :

1. **Scripts de création SQL numérotés et automatisés avec Flyway** (Retenu).
2. Laisser le framework Java deviner et modifier les tables tout seul à chaque lancement.
3. Demander à chaque développeur d'exécuter manuellement des commandes dans sa base.

Notre choix et pourquoi : Laisser un outil automatique modifier la base de données à l'aveugle est une cause majeure de pannes et de pertes de données. Avec Flyway, chaque changement de structure est écrit dans un script SQL clair, daté et vérifié. Au démarrage, l'application lit ces scripts dans l'ordre et met à jour la base sans intervention humaine, assurant une installation infaillible sur n'importe quel ordinateur.

2.1.9 Axe 9 : Conteneurisation et Gestion des Secrets

La question : Comment permettre de démarrer tout le projet en un clic sans risque de fuite de mots de passe ?

Options étudiées :

1. **Lancement complet via Docker Compose et externalisation des mots de passe dans un fichier de configuration protégé (.env.example)** (Retenu).
2. Obliger à installer manuellement PostgreSQL, Java et Node.js sur la machine hôte.
3. Écrire les mots de passe et clés secrètes directement dans le code source.

Notre choix et pourquoi : Docker Compose permet de réunir la base de données, le backend et le frontend dans des boîtes virtuelles isolées (conteneurs) et de tout lancer avec une seule commande (`docker compose up`). Par ailleurs, pour garantir la sécurité, aucun mot de passe n'est écrit « en dur » dans le code : ils sont placés dans un fichier local (`.env`) rigoureusement ignoré par Git, ce qui évite toute fuite accidentelle sur Internet tout en proposant un modèle prêt à l'emploi (`.env.example`).

2.2 Modèle de Données et Schéma Relationnel

Le schéma de la base de données est composé de trois tables principales :

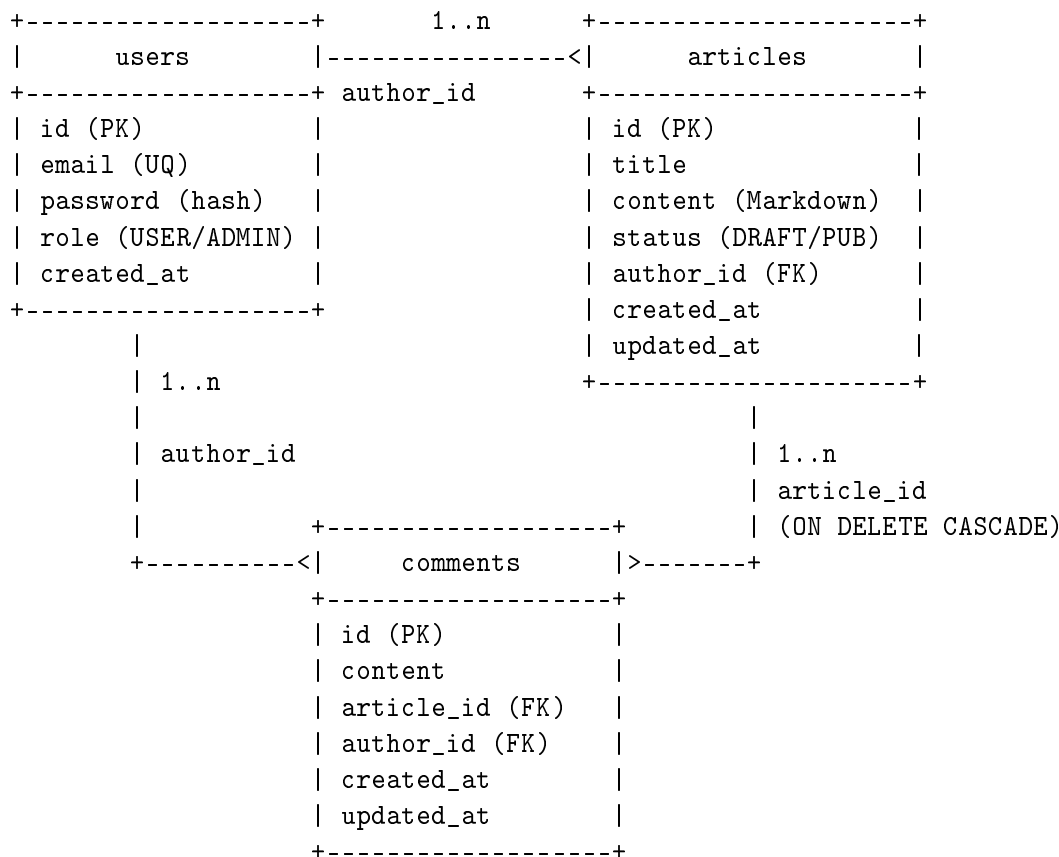


FIGURE 1 – Diagramme Relationnel de la Base de Données PostgreSQL

- **users** : Stocke l'adresse email (contrainte d'unicité), le mot de passe haché par l'algorithme BCrypt, et le rôle (`ROLE_USER` ou `ROLE_ADMIN`).
- **articles** : Stocke le titre, le contenu rédigé en Markdown, le statut (`DRAFT` ou `PUBLISHED`), la clé étrangère vers l'auteur (`author_id`), et les timestamps de création et mise à jour.
- **comments** : Stocke le texte du commentaire et les clés étrangères (`article_id`, `author_id`), avec suppression en cascade (`ON DELETE CASCADE`).

2.3 Matrice de Sécurité et Droits d'Accès

La sécurité combine deux niveaux complémentaires : le contrôle d'accès basé sur les rôles (*Role-Based Access Control*) et le contrôle au niveau des instances de ressources (*Resource-Level Security*).

TABLE 1 – Matrice Synthétique des Droits d'Accès par Rôle

Opération / source	Res-	Visiteur Anonyme	Auteur Connecté	Administrateur
Authentification		Inscription Connexion	/ Session active	Session active
Articles publiés		Consultation libre	Consultation libre	Modération & Dépublication
Brouillons (DRAFT)		Inaccessibles	Création, édition & suppression de ses brouillons	Consultation globale, édition & publication
Commentaires sur publiés	sur	Lecture seule	Ajout, édition & suppression de ses écrits	Ajout & modération de tous les commentaires
Commentaires sur brouillons	sur	× (Interdit)	× (Interdit)	× (Interdit)

2.4 Architecture Logicielle en Couches

L'application respecte les principes de l'architecture en couches (*Layered Architecture*) :

1. **Couche Présentation (Controllers)** : Définition des endpoints REST, validation des requêtes DTOs via Bean Validation (`@Valid`, `@NotBlank`, `@Email`), sérialisation JSON et documentation OpenAPI.
2. **Couche Sécurité & Filtres** : Filtre d'authentification interceptant les requêtes, déchiffrant le JWT et alimentant le contexte de sécurité. Les évaluateurs d'autorisation appliquent les règles de propriété des ressources.
3. **Couche Métier (Services)** : Implémentation des règles fonctionnelles, vérification de l'intégrité (statut DRAFT vs PUBLISHED) et gestion transactionnelle (`@Transactional`).
4. **Couche Persistance (Repositories)** : Interfaces d'accès aux données exploitant des requêtes JPQL sécurisées pour le filtrage contextuel et la pagination.
5. **Couche Infrastructure** : Initialisation du schéma SQL et conteneurisation reproductible via `docker-compose.yml`.

2.5 Validation et Tests Automatisés du Backend

Afin de garantir la robustesse des règles de gestion et la stricte étanchéité de la sécurité au niveau ressource, une suite de 18 tests automatisés a été implémentée avec JUnit 5, Mockito et MockMvc.

3 Partie II : Architecture et Réalisation du Frontend

3.1 Choix de Conception du Client Web

Cette sous-section détaille les 4 décisions structurantes du client web Angular, également vulgarisées pour en comprendre les raisons pratiques.

TABLE 2 – Synthèse Concise des 18 Tests Automatisés du Backend

Domaine	Type	Règles et Scénarios Validés	Tests	Statut
Authentification	Unitaire	Inscription nominale, unicité d'email (409), login valide et génération JWT	3	✓OK
Articles	Unitaire	Création en brouillon forcé, droits auteur, verrouillage publié, modération admin, workflow publier/dépublier, cascade	7	✓OK
Commentaires	Unitaire	Ajout sur publié, interdiction sur brouillon, modification/suppression auteur, modération admin	7	✓OK
Sécurité Globale	Intégration	Scénario complet en 15 étapes (création compte, isolation brouillon, publication admin, cascade)	1	✓OK
Total de la suite automatisée (exécutable via mvn test)			18	100%

3.1.1 Axe 10 : Solution de Style et Design Visuel

La question : Comment concevoir une interface agréable, moderne et parfaitement lisible sur smartphone comme sur ordinateur ?

Options étudiées :

1. **Tailwind CSS (Mise en forme utilitaire moderne et sur-mesure)** (Retenu).
2. Bootstrap 5 (Composants prédéfinis classiques).
3. Angular Material (Design standardisé Google).
4. CSS pur écrit entièrement à la main sans outil d'aide.

Notre choix et pourquoi : Plutôt que d'écrire des centaines de lignes de CSS complexes ou de subir des composants rigides difficiles à personnaliser, Tailwind CSS permet de styliser chaque élément directement avec des mots-clés simples et visuels. Cela nous a permis de concevoir des composants sur-mesure très soignés (badges ambre pour les brouillons, badges vert émeraude pour les articles publiés, formulaires épurés), tout en garantissant une application légère et très rapide à charger.

3.1.2 Axe 11 : Gestion des Pannes Réseau et Arrêt du Serveur

La question : Que doit voir l'utilisateur si le serveur backend s'arrête ou ne répond plus ?

Options étudiées :

1. **Approche moderne et discrète : Toast flottant non-bloquant avec compte à rebours de reconnexion et bouton d'action manuelle** (Retenu).
2. Bannière rouge bloquante occupant tout le haut de page.
3. Redirection brutale vers une page d'erreur blanche (503 Service Unavailable).

Notre choix et pourquoi : Si le serveur rencontre une coupure temporaire, rediriger brutalement l'utilisateur vers une page d'erreur ferait perdre tout le texte qu'il était en train d'écrire. À l'inverse, ne rien afficher le laisserait dans l'incompréhension face à des boutons qui ne répondent plus. Nous avons conçu une notification flottante moderne : elle prévient calmement l'utilisateur sans masquer son travail, décompte les secondes avant une nouvelle tentative automatique, et propose un bouton pour réessayer immédiatement dès que la connexion est rétablie.

3.1.3 Axe 12 : Routage et Communication avec l'API

La question : Comment faire dialoguer le site web Angular et l'API Spring Boot sans rencontrer de blocages du navigateur ?

Options étudiées :

1. **Serveur Nginx comme porte d'entrée unique sur le port standard 4200, distribuant le site et relayant les requêtes vers l'API en coulisses** (Retenu).
2. Lancer deux serveurs totalement distincts et configurer des autorisations complexes entre domaines (CORS).

Notre choix et pourquoi : Par sécurité, les navigateurs internet bloquent fréquemment les échanges lorsqu'un site web tente d'appeler un serveur situé sur une adresse ou un port différent (les fameuses erreurs CORS). Nginx résout ce problème à la racine en agissant comme un « guichet unique » sur le port habituel d'Angular (4200) : pour l'internaute, tout provient du même endroit. Nginx affiche le site web et transmet de manière transparente les demandes de données au serveur Spring Boot en arrière-plan.

3.1.4 Axe 13 : Rédaction et Mise en Page des Articles

La question : Comment permettre aux auteurs d'enrichir le texte de leurs articles sans risquer d'introduire des failles de sécurité ?

Options étudiées :

1. **Format Markdown léger avec onglet d'aperçu en direct et conversion sécurisée** (Retenu).
2. Texte brut sans aucune mise en forme possible.
3. Éditeur de traitement de texte complexe générant directement du code HTML (WYSIWYG).

Notre choix et pourquoi : Permettre aux utilisateurs d'écrire du code HTML brut dans un formulaire est extrêmement dangereux : un pirate pourrait y glisser des scripts malveillants pour voler les comptes des lecteurs (attaques XSS). Le format Markdown est la solution idéale : il permet d'ajouter simplement du gras, des titres, des citations et du code informatique avec quelques symboles faciles à retenir (#, **), tandis que notre application transforme ce texte en un affichage propre, lisible et garanti sans risque de sécurité.

3.2 Architecture Logicielle et Découpage Modulaire

Le projet frontend est structuré selon les standards modernes d'Angular 18 (*Standalone Components, Signals*) :

- **src/app/core/** : Éléments transversaux instanciés sous forme de singletons :
 - **Services Réactifs** : `AuthService` (gestion du token et signaux), `ArticleService` (appels REST paginés), `CommentService`, et `NetworkStatusService` (suivi de connectivité).
 - **Guards Fonctionnels** : `authGuard` (protection auteur) et `adminGuard` (protection administration).
 - **Intercepteurs HTTP** : Injection automatique du jeton (`jwtInterceptor`) et gestion globale des erreurs (`errorInterceptor`).
 - **Modèles TypeScript** : Interfaces de données typées (`Article`, `User`, `Comment`, etc.).
- **src/app/features/** : Composants métier dédiés aux différentes vues :

- **Flux Public** (`ArticleListComponent`) : Affichage en grille responsive des articles publiés, barre de recherche textuelle et pagination.
- **Détail et Commentaires** (`ArticleDetailComponent`) : Lecture avec rendu Markdown, métadonnées et fil de commentaires avec modération contextuelle.
- **Espace Auteur** (`MyDraftsComponent`) : Liste réservée aux articles au statut DRAFT appartenant au rédacteur connecté.
- **Édition / Création** (`ArticleFormComponent`) : Formulaire réactif avec double onglet « Rédiger » et « Prévisualiser » pour un contrôle en temps réel du rendu Markdown.
- **Dashboard Administration** (`AdminDashboardComponent`) : Console de modération complète avec métriques, filtres dynamiques, prévisualisation modale, et boutons de workflow (**Publier**, **Dépublier**, etc.).
- **Authentification** (`LoginComponent` & `RegisterComponent`) : Formulaires sécurisés avec boutons d'amorçage rapide pour les profils de test (Admin et Auteur).
- `src/app/shared/` : Composants et utilitaires réutilisables :
 - `NavbarComponent` : Barre de navigation réactive s'adaptant instantanément à l'état de connexion et aux privilèges de l'utilisateur.
 - `BackendErrorBannerComponent` : Toast flottant non intrusif affiché lors d'une indisponibilité du serveur backend, avec bouton de reconnexion.
 - `MarkdownPipe` : Tube de transformation textuelle convertissant la syntaxe Markdown en balises HTML sécurisées.

3.3 Écrans et Parcours Utilisateur

L'application couvre l'ensemble des parcours fonctionnels d'un blog collaboratif :

1. **Visiteur Anonyme** : Consulte le flux des articles publiés sur `/articles`, effectue des recherches textuelles, lit les détails d'un article et ses commentaires. Il peut créer un compte sur `/register` ou s'identifier sur `/login`.
2. **Auteur Authentifié** (`ROLE_USER`) : Rédige des articles sur `/articles/nouveau` (créés en DRAFT). Il dispose de son espace `/mes-brouillons` pour retrouver ses articles en cours, les modifier et les supprimer. Il peut commenter les articles publiés.
3. **Administrateur** (`ROLE_ADMIN`) : Accède au dashboard de supervision `/admin`. Il visualise l'ensemble des articles du site, filtre par statut, ouvre la prévisualisation modale, et utilise les boutons d'action pour publier un brouillon ou dépublier un article. Il bénéficie également d'un pouvoir de modération absolue sur tous les commentaires.

4 Partie III : Déploiement, Gestion des Secrets et Conclusion

4.1 Orchestration Conteneurisée et Isolation des Secrets

L'ensemble de la plateforme est orchestré via un fichier `docker-compose.yml` unique pilotant les 3 conteneurs sur un réseau interne :

1. `miniblog-postgres` : Base PostgreSQL 16 Alpine avec healthcheck automatisé.
2. `miniblog-backend` : API Spring Boot 3 compilée en conteneur multi-stage (JRE 21 slim).
3. `miniblog-frontend` : Serveur Nginx Alpine servant l'application Angular compilée et assurant le reverse-proxying vers l'API.

Gestion des Secrets d'Environnement : Les informations sensibles (mots de passe de la base de données, clé secrète HMAC-SHA du JWT, identifiants du compte administrateur) sont gérées selon les meilleures pratiques DevSecOps :

- Un gabarit documenté `.env.example` est versionné dans le dépôt Git comme modèle de référence.

- Un fichier local `.env` contient les secrets actifs chargés par Docker Compose.
- Le fichier `.gitignore` bloque strictement tout envoi de fichiers d'environnement sur le dépôt distant.
- L'application web est exposée sur le port conventionnel Angular 4200 (4200:80).

4.2 Conclusion et Bilan de Réalisation

La plateforme **Mini-Blog** dispose d'une implémentation fullstack complète, opérationnelle et rigoureusement documentée :

- **Excellence Backend** : Architecture Spring Boot 3 en Java 21, persistance PostgreSQL 16 avec migrations Flyway, hachage BCrypt, jetons JWT stateless et sécurité granulaire par ressource.
- **Couverture de Tests** : Suite automatisée de 18 tests unitaires et d'intégration validant 100% de la matrice des droits et des règles d'étanchéité éditoriale.
- **Expérience Frontend** : Application Angular 18 moderne, performante et réactive exploitant les Signals, un design system moderne avec Tailwind CSS, et une tolérance aux pannes réseau.
- **Déploiement Reproductible** : Orchestration conteneurisée via Docker Compose, isolation étanche des secrets d'environnement via `.env.example`, et reverse-proxying Nginx sur le port conventionnel 4200.